

High-Level Synthesis for Scheduling Pipelined Circuits: A Small List-Based Modulo Scheduler with UPPAAL and VHDL Verification

Md Ratul Ahmed Alvi

Department of Electrical Engineering
Hochschule Hamm-Lippstadt (HSHL)
Lippstadt, Germany
md-ratul-ahmed.alvi.@stud.hshl.de

Abstract—High-level synthesis translates behavioral descriptions into register-transfer level hardware. One important step is scheduling, because the selected start time of each operation determines resource use, latency, and throughput. This paper presents a small list-based modulo scheduler written in C++ for pipelined loops. The scheduler considers resource-constrained and recurrence-constrained lower bounds on the initiation interval (II). Three examples are studied: a 3-tap FIR filter, an accumulation loop, and a single-multiply delay line. The experiments demonstrate resource-limited, recurrence-limited, and $II = 1$ scheduling cases. To extend the evaluation beyond the C++ scheduler, the FIR schedule is also represented in UPPAAL, where timing, resource constraints, deadlock freedom, and completion are checked for three overlapping iterations. A corresponding VHDL implementation is then simulated to verify the numerical FIR behavior. Together, the three stages show how analytical scheduling, formal model checking, and RTL simulation can provide different but complementary forms of verification.

Index Terms—high-level synthesis, modulo scheduling, list scheduling, initiation interval, pipeline, data-flow graph

I. INTRODUCTION

Designing a circuit directly in VHDL or Verilog gives the designer detailed control, but it also requires time and hardware-specific experience. High-level synthesis (HLS) reduces part of this effort by converting a behavioral description into an RTL implementation. The main HLS steps include scheduling, allocation, and binding [1]. Scheduling decides when each operation starts and therefore has a direct effect on performance and resource usage.

Loops are common in signal processing and control applications. If every iteration is executed completely before the next one starts, the resulting throughput can be low. Loop pipelining improves throughput by overlapping successive iterations. The time between the start of two iterations is called the initiation interval (II). A smaller II normally means a higher steady-state throughput.

This work studies a small list-based modulo scheduler. The purpose is not to replace a production HLS tool, but to demonstrate the main scheduling constraints in a form that can be followed and tested by a student. The paper first introduces

the mathematical model, then describes the scheduler, and finally evaluates three small examples.

II. BACKGROUND

A. Data-Flow Graph

A loop body can be represented by a data-flow graph $G = (V, E)$. Each node $v \in V$ is an operation, and each edge $e = (u, v, d)$ represents a dependency from operation u to operation v . The value d is the iteration distance. An edge with $d = 0$ is inside one iteration, while $d > 0$ is a loop-carried dependency.

For a target initiation interval II , a valid modulo schedule must satisfy

$$t(v) \geq t(u) + \lambda(u) - d \cdot II, \quad (1)$$

where $t(v)$ is the start time and $\lambda(u)$ is the latency of operation u . Equation (1) is important because it handles both normal and loop-carried dependencies.

B. Resource and Recurrence Bounds

The resource-constrained initiation interval estimates how often the required operations can be started with the available hardware. If every functional unit can accept one new operation per cycle, the lower bound is

$$ResII = \max_r \left\lceil \frac{N_r}{M_r} \right\rceil, \quad (2)$$

where N_r is the number of operations of resource type r in one iteration and M_r is the number of available units. If a unit is not fully pipelined, its own initiation interval must also be included in the resource calculation.

A loop-carried cycle with total latency D and iteration distance L gives the recurrence bound

$$RecII = \left\lceil \frac{D}{L} \right\rceil. \quad (3)$$

The scheduler cannot use an II below

$$II_{\min} = \max(ResII, RecII). \quad (4)$$

These are lower bounds. A heuristic scheduler may still require a larger II if it cannot find a feasible placement.

C. Latency and Throughput

Latency is the time from the start of one iteration until its result is available. Throughput describes how frequently results are produced after the pipeline has filled. For one result per iteration,

$$\text{Throughput} = \frac{1}{II}. \quad (5)$$

A pipeline can therefore improve throughput without reducing the latency of one individual iteration.

III. LIST-BASED MODULO SCHEDULER

The implementation uses a simple iterative approach inspired by modulo scheduling methods [2], [3]. Operations are first ordered by their ASAP times. For each target II , the scheduler tries to place every operation at the earliest cycle that satisfies both dependency and modulo-resource constraints.

- 1) Compute $ResII$, $RecII$, and $II_{\min}$.
- 2) Start with $II = II_{\min}$.
- 3) Compute an ASAP-based priority order.
- 4) Place each operation at the earliest valid cycle.
- 5) Check all dependencies using Eq. (1).
- 6) If the schedule is invalid, increase II and repeat.

The resource table is indexed by $c \bmod II$, because operations from later iterations repeat in the same modulo slots. Listing 1 shows the main idea. The actual program also checks dependencies before accepting a cycle.

```

1 for (int id : order) {
2   int c = asap[id];
3   int searched = 0;
4   const int SEARCH_LIMIT = maxSearchCycles;
5
6   while ((!dependenciesSatisfied(id, c, II, schedule)
7         || resourceUsed(c % II, ops[id].type)
8         >= available(ops[id].type))
9         && searched < SEARCH_LIMIT) {
10    ++c;
11    ++searched;
12  }
13
14  if (searched == SEARCH_LIMIT)
15    return FAILURE; // try a larger II
16
17  schedule[id] = c;
18  reserve(c % II, ops[id].type);
19 }
```

Listing 1. Simplified greedy placement loop.

The search is bounded by a configurable cycle limit. If no valid placement is found within this limit, the current scheduling attempt returns failure and the outer loop retries with a larger II . This avoids an unbounded search when the current modulo slots cannot provide a feasible placement.

The program represents each operation with a type, latency, dependency list, and iteration distance. The minimum information is illustrated in Listing 2.

```

1 struct Edge {
2   int source;
3   int distance;
4 };
```

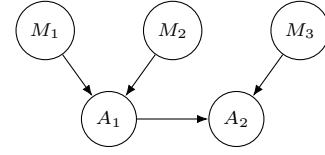


Fig. 1. Data-flow graph of the 3-tap FIR loop.

TABLE I
FIR SCHEDULE FOR ONE ITERATION, $II = 3$.

Operation	Start cycle	Resource
M_1	0	Multiplier
M_2	1	Multiplier
M_3	2	Multiplier
A_1	3	Adder
A_2	4	Adder

```

5 struct Op {
6   Resource type;
7   int latency;
8   vector<Edge> deps;
9 };
10
```

Listing 2. Operation representation used by the scheduler.

This implementation is deliberately small. Its runtime depends on the number of operations, the number of tested II values, and how far the cycle search moves before a free valid slot is found. Therefore, a fixed $O(n \log n)$ claim would be too strong for the complete implementation. Sorting the priority list costs $O(n \log n)$, but the repeated feasibility search adds further work.

IV. EXPERIMENTS

All examples use fully pipelined functional units: an operation may have a latency greater than one, but the same unit can accept a new operation every cycle. This assumption is necessary to interpret Eq. (2) correctly.

A. FIR Filter: Resource-Limited Case

The first loop implements

$$y[n] = ax[n] + bx[n-1] + cx[n-2]. \quad (6)$$

The delay elements are initialized to zero, so $x[-1] = x[-2] = 0$ for the first input sample.

It contains three multiplications and two additions. One multiplier and two adders are available. Multiplier latency is two cycles and adder latency is one cycle. The graph has no loop-carried dependency.

The resource bound is

$$ResII = \left\lceil \frac{3}{1} \right\rceil = 3, \quad RecII = 0. \quad (7)$$

The scheduler finds $II = 3$, which reaches the lower bound. Table I shows one modulo schedule. The three multiplier starts occupy the three modulo slots, while the two additions use separate adder resources.

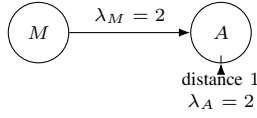


Fig. 2. Accumulation loop with a loop-carried recurrence.

TABLE II
ACCUMULATION SCHEDULE, $II = 2$.

Iteration	M start	A start	Result
0	0	2	4
1	2	4	6
2	4	6	8
3	6	8	10

The result latency is five cycles, while a new iteration starts every three cycles. The steady-state throughput is therefore $1/3$ result per cycle.

B. Accumulation: Recurrence-Limited Case

The second loop is

$$sum += x[i] \cdot coeff. \quad (8)$$

One multiplier and one adder are available. Both units are fully pipelined. Multiplier latency is two cycles. To create a clear recurrence-limited example, the accumulator addition is modeled with latency two cycles. Its result is used by the next iteration, giving a recurrence with $D = 2$ and $L = 1$.

The bounds are

$$ResII = 1, \quad RecII = \left\lceil \frac{2}{1} \right\rceil = 2. \quad (9)$$

The scheduler therefore starts at $II_{\min} = 2$ and finds a feasible schedule. Table II shows the repeated start pattern.

This is a genuine recurrence-limited result. Adding more multipliers or adders does not reduce II below two unless the recurrence is restructured or its latency is reduced.

C. Delay Line: II Equal to One

The third loop computes

$$y[n] = x[n] \cdot coeff. \quad (10)$$

It contains one multiplication per iteration, two multipliers are available, and there is no loop-carried dependency. Thus,

$$ResII = 1, \quad RecII = 0, \quad II = 1. \quad (11)$$

The multiplier latency remains two cycles, but a new iteration begins every cycle. After the pipeline is filled, one result is produced per cycle.

D. Summary

Table IV compares the three cases. The results match the intended lower bound in each example.

TABLE III
DELAY-LINE SCHEDULE, $II = 1$.

Iteration	Multiply start	Result cycle
0	0	2
1	1	3
2	2	4
3	3	5
4	4	6

TABLE IV
SUMMARY OF EXPERIMENTS.

Example	$ResII$	$RecII$	II
3-tap FIR	3	0	3
Accumulation	1	2	2
Delay line	1	0	1

E. Validation Method

Each experiment was checked in three steps. First, the analytical lower bounds were calculated using Eqs. (2)–(4). Second, the C++ scheduler generated start times for all operations. Third, an independent validation routine re-checked the completed schedule after generation. This validator is separate from the checks used during greedy placement and checks every dependency and every modulo-resource slot again.

For each edge (u, v, d) , the validator evaluates Eq. (1). For each resource type, it also counts the operation starts mapped to every $c \bmod II$ slot. A schedule is accepted only when all dependency inequalities hold and no slot uses more units than are available. These checks are small, but they prevent the scheduler from reporting a result that only appears feasible.

The experiments were deliberately chosen so that one expected limit was clear in each case. This made it possible to compare the observed II with a known analytical expectation instead of reporting the program output without a reference value.

V. HARDWARE/SOFTWARE CO-DESIGN VIEW

The experiment can also be understood as a small hardware/software co-design workflow. The scheduling algorithm runs as software and receives a behavioral model, operation latencies, dependency information, and resource limits. Its output is a cycle-level schedule. That schedule then guides the hardware architecture by showing when multipliers and adders are required and which operations may share the same unit.

The software side performs design-space reasoning: it tests the lower bounds, searches for valid start times, and reports whether the current resource budget is sufficient. The hardware side is represented by the selected number of functional units and their pipeline behavior. For example, the FIR result shows that one multiplier forces three multiplication start slots per iteration. This information can be used to decide whether the extra area of a second multiplier is justified by a lower II .

The separation is useful because hardware changes can be evaluated first at the scheduling level. However, the schedule is still only a model. A complete flow would also generate

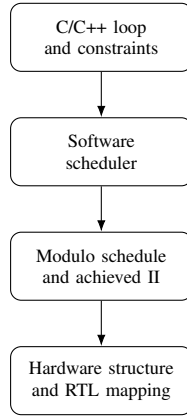


Fig. 3. Simplified hardware/software co-design flow used in this work.

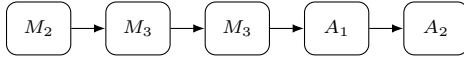


Fig. 4. Abstract UPPAAL operation sequence for one FIR iteration.

RTL, synthesize it for an FPGA, and compare the predicted II with simulation and timing reports.

For the verified FIR configuration, one fully pipelined multiplier gives $ResII = 3$ because three multiplications must be started in every iteration. Increasing the multiplier count would change the analytical resource bound, but a new scheduling run would still be required before claiming a lower achieved II. This is the practical co-design decision in this example: additional hardware may improve throughput, but the improvement must be checked against dependencies and the complete schedule rather than assumed from resource count alone.

VI. UPPAAL MODELING AND VERIFICATION

The C++ scheduler gives start times and an achieved initiation interval, but this alone does not formally check properties such as deadlock freedom or resource occupancy. For this reason, the FIR schedule was also represented as a small UPPAAL model [5]. The purpose of this model is not to reproduce the complete RTL circuit. Instead, it represents the timing order of the scheduled operations and checks whether overlapping iterations respect the selected resource limits.

A. Abstract FIR Automaton

One FIR iteration is abstracted by the sequence

$$M_1 \rightarrow M_2 \rightarrow M_3 \rightarrow A_1 \rightarrow A_2 \quad (12)$$

where M_2 , M_3 , and M_3 are the three multiplications and A_1 and A_2 are the two additions. The multiplier operations are issued in three consecutive cycles. A new iteration is released every three cycles, matching the scheduler result $II = 3$.

Three automaton instances are used to represent three overlapping iterations. Their release times are separated by three cycles. Shared counters `mult_users` and `add_users` represent current resource occupancy. The model prevents the number of active users from exceeding the available hardware resources.

TABLE V
UPPAAL VERIFICATION PROPERTIES.

Property	Purpose
$A[]$ not deadlock	No reachable deadlock
$A[]$ mult_users ≤ 1	One multiplier limit
$A[]$ add_users ≤ 2	Two-adder limit
$A\langle \rangle$ completed == 3	All three iterations can complete
$A\langle \rangle$ I0.Done and I1.Done and I2.Done	Every modeled instance reaches Done

B. Verification Properties

The verifier checks the properties summarized in Table V. The first property checks deadlock freedom. The next two check the multiplier and adder limits. The final two confirm that all three modeled iterations can finish.

For the tested model, all five queries are satisfied. This gives additional evidence that the abstract $II = 3$ FIR schedule is free from deadlock, respects the modeled resource limits, and allows all three overlapping iterations to finish.

The current UPPAAL model checks exactly three overlapping FIR iterations. It is therefore a bounded verification example and not a parametric proof for an arbitrary number of iterations. Extending the model to a generalized parameterized version is left as future work.

VII. VHDL IMPLEMENTATION AND SIMULATION

The FIR example was also implemented in VHDL to check the numerical RTL behavior. The design uses one shared fully pipelined multiplier and two iteration-storage slots. A new input sample is accepted every three clock cycles, corresponding to the scheduled $II = 3$.

The input interface uses a valid/ready handshake. A sample is accepted only when `in_valid='1'` and the design is ready for a new iteration. In the RTL controller, phase selects which of the three FIR multiplications is issued in the current cycle. The signal `next_slot` alternates between two storage slots so that values belonging to overlapping iterations remain separated.

```

1 in_ready <= '1' when phase = 0 else '0';
2
3 case phase is
4   when 0 =>
5     if in_valid = '1' then
6       active_slot <= next_slot;
7       sample_0(next_slot) <= x_in;
8       sample_1(next_slot) <= delay_1;
9       sample_2(next_slot) <= delay_2;
10      next_slot <= 1 - next_slot;
11    end if;
12
13   when 1 =>
14     -- issue multiplication with coeff_b
15
16   when others =>
17     -- issue multiplication with coeff_c
18 end case;
  
```

Listing 3. Simplified FIR input and phase control.

The testbench uses coefficients (1, 2, 3) and the input sequence (10, 20, 30, 40). The delay elements are initially zero, so $x[-1] = x[-2] = 0$. Therefore,

TABLE VI
VHDL FIR SIMULATION RESULTS.

n	$x[n]$	Expected $y[n]$	Simulated $y[n]$
0	10	10	10
1	20	40	40
2	30	100	100
3	40	160	160

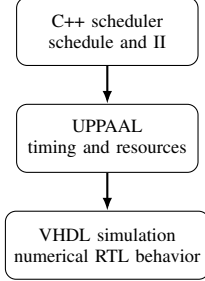


Fig. 5. Three-stage evaluation flow used in this work.

$$y[n] = x[n] + 2x[n-1] + 3x[n-2]. \quad (13)$$

The expected and simulated results are shown in Table VI.

The self-checking testbench compares each valid output with its expected value. The four results 10, 40, 100, and 160 are produced correctly. This simulation checks the arithmetic behavior of the RTL implementation, while the UPPAAL model checks abstract timing and resource properties.

The three stages have different purposes. The C++ scheduler constructs a cycle-level schedule, UPPAAL checks the abstract scheduling properties, and VHDL simulation checks whether the corresponding RTL produces the expected numerical results.

VIII. DISCUSSION

The examples show why the dominant bound must be identified before changing the hardware. In the FIR filter, the resource bound dominates. Adding a second multiplier could lower $ResII$, although a new schedule would still be required. In the accumulation loop, the recurrence dominates. Additional units alone do not change the two-cycle feedback path. In the delay line, neither bound is greater than one.

The accumulation example also shows an important modeling lesson. An ordinary dependency from a multiplication to an addition inside the same iteration does not by itself create $RecII$. A recurrence bound requires a loop-carried dependency. The latency around that loop-carried cycle and its iteration distance determine Eq. (3).

The implementation remains a teaching model. Production modulo schedulers handle memory dependencies, control flow, chaining, operator-specific initiation intervals, and schedule-length optimization. Exact formulations can prove optimal IIs for many practical loops, but they are more complex than the list-based approach used here [4]. Heuristic and SDC-based

approaches offer other trade-offs between scheduling quality and runtime [3].

The three tools also check different kinds of possible errors. The C++ scheduler determines whether a cycle-level operation schedule can be constructed under the selected resource and dependency constraints. UPPAAL checks the abstract timing, resource occupancy, completion, and deadlock properties, but it does not verify the numerical arithmetic. VHDL simulation checks the numerical RTL behavior and signal-level timing, but it does not by itself prove the abstract scheduling properties checked in UPPAAL. For this reason, the three stages provide complementary evidence rather than repeating the same verification.

IX. CONCLUSION

This paper presented a small list-based modulo scheduler for pipelined loops in HLS. The scheduler starts from the maximum of the resource and recurrence bounds and increases II only when a feasible schedule cannot be found.

Three corrected examples demonstrated different limits. The FIR filter reached $II = 3$ because one multiplier had to start three multiplications per iteration. The accumulation loop reached $II = 2$ because of a two-cycle loop-carried recurrence. The delay line reached $II = 1$ because neither resources nor dependencies required a larger interval. The main result is that operation latency, functional-unit initiation capability, and loop-carried dependence must be modeled separately. Without this distinction, an observed II can easily be attributed to the wrong cause.

In addition to the analytical and C++ scheduling results, the FIR schedule was checked with an UPPAAL model for three overlapping iterations. The model satisfied the tested deadlock, multiplier-resource, adder-resource, and completion properties. A corresponding VHDL implementation was also simulated with coefficients (1, 2, 3) and input samples (10, 20, 30, 40), producing the expected outputs 10, 40, 100, and 160. These results show the complementary role of the three stages: scheduling constructs the operation timing, UPPAAL checks abstract timing and resource properties, and VHDL simulation checks numerical RTL behavior.

REFERENCES

- [1] G. De Micheli, *Synthesis and Optimization of Digital Circuits*. New York, NY, USA: McGraw-Hill, 1994.
- [2] B. R. Rau, "Iterative modulo scheduling: An algorithm for software-pipelining loops," in *Proc. 27th Annu. Int. Symp. Microarchitecture*, 1994, pp. 63–74.
- [3] Z. Zhang and B. Liu, "SDC-based modulo scheduling for pipeline synthesis," in *Proc. IEEE/ACM Int. Conf. Computer-Aided Design*, 2013, pp. 211–218.
- [4] J. Oppermann, M. Reuter-Oppermann, L. Sommer, A. Koch, and O. Sinn, "Exact and practical modulo scheduling for high-level synthesis," *ACM Trans. Reconfigurable Technol. Syst.*, vol. 12, no. 2, pp. 1–26, 2019.
- [5] G. Behrmann, A. David, and K. G. Larsen, "A tutorial on UPPAAL," in *Formal Methods for the Design of Real-Time Systems*, Lecture Notes in Computer Science, vol. 3185. Berlin, Germany: Springer, 2004, pp. 200–236.